
University of Engineering and Technology, Lahore

Department of Artificial Intelligence

DSA LAB : 02

Implementation of Arrays

Name	Umama Ume Amen
Registration No.	2025-AI-264
Section	A (Afternoon)
Instructor	Ma'am Husna

Experiment 2

Implementation of Arrays

OBJECTIVE:

- Declare and initialize arrays in C++.
- Understand logical size and capacity.
- Perform array operations: Traverse, Search, Insert, Delete, and Update.
- Handle overflow and invalid index conditions

INTRODUCTION:

An array is a linear data structure that stores elements of the same data type in contiguous memory locations. Arrays provide fast direct access using index values. However, insertion and deletion in the middle require shifting of elements. In this lab, we will implement a simple array-based list manually using:

- A fixed array
- A variable *n* to track the current number of elements

ARRAY DECLARATION AND ASSIGNMENT:

Array declaration requires specifying the data type and capacity. Logical size (*n*) keeps track of the number of elements currently stored. For example the code below stores even numbers from 60 to 88 in an Array and display 'n'.

```
#include <iostream>
using namespace std;

const int CAPACITY = 100;

int arr[CAPACITY];
int n = 0;

int main() {

    for (int val = 60; val < 89; val+=2)
    {
        arr[n] = val;
        n++;
    }
    cout<<"n= "<<n<<"\n";

    return 0;
}
```

(1)

TRAVERSE OPERATION

Traversal means visiting each element of the array sequentially.

Used for:

- Printing values
 - Calculating sum
 - Finding max/min
 - Applying operations on each element
- Traversal requires a loop from 0 to n-1.

```
void traverse() {
    if (n == 0) {
        cout << "Array is empty\n";
        return;
    }

    cout << "Array elements: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

(2)

SEARCH OPERATION

Search means finding the position of a value. Since array is unsorted, we use Linear Search:

- Start from index 0
- Compare each element
- Stop if found

Worst case → check all elements.

```
#include <iostream>
using namespace std;

const int CAPACITY = 100;

int arr[CAPACITY]; int
n = 0;

int search(int value) {

    for (int i = 0; i < n; i++) {
        if (arr[i] == value)
            return i;
    }

    return -1; // not found
```

```

} int
main() {
int index;
    for (int val = 60; val < 89; val+=2)
    {
        arr[n] = val;        n++;
    }
    index= search(65);
    cout<<index;

    return 0;
}

```

(3)

INSERT OPERATION

Insertion at end places element at index n. Insertion at index requires shifting elements to the right.

Function for insertion at the end:

```

void insertEnd(int value) {
if (n == CAPACITY) {
    cout << "Overflow!";
return;
}
    arr[n] = value;
n++;
}

```

(4)

Function for insertion at a specific index:

```

void insertAt(int index, int value) {
    if (index < 0 || index > n || n == CAPACITY) {
        cout << "Invalid operation";
return;
    }
    for (int i = n; i > index;
i--) {
        arr[i] = arr[i - 1];
    }
    arr[index] = value;
n++;
}

```

(5)

DELETE OPERATION

Deletion removes element and shifts remaining elements to the left.

```

    for (int i = index; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
n--;
}

```

(6)

UPDATE OPERATION

Update modifies the value at a specific index without shifting.

```
void update(int index, int newValue) {  
    if (index < 0 || index >= n) {  
        cout << "Invalid index";  
        return;  
    }  
    arr[index] = newValue;  
}
```

(7)

LAB TASKS:

1. Run all operations described in codes 1-7 and record the output.
2. Construct the code implement the following:
 - i) Function to find maximum value.
 - ii) Function to reverse the array.
 - iii) Function to remove all occurrences of a value.

Completed C++ code and execution evidence

```
exercise1.cpp > deleteAt(int)
1  #include <iostream>
2  using namespace std;
3  const int CAPACITY = 100;
4  int arr[CAPACITY];
5  int n = 0;
6  void traverse() // traverse operation is used here
7  {
8      if (n == 0) {
9          cout << "Array is empty\n";
10         return;
11     }
12     cout << "Array elements: ";
13     for (int i = 0; i < n; i++) {
14         cout << arr[i] << " ";
15     }
16     cout << endl;
17 }
18 int search(int value) // we are searching position of a value
19 {
20     for (int i = 0; i < n; i++) {
21         if (arr[i] == value)
22             return i;
23     }
24     return -1;
25 }
26 void insertEnd(int value) // inserting at extyreme ends
27 {
28     if (n == CAPACITY) {
29         cout << "Overflow!\n";
30         return;
31     }
32     arr[n] = value;
33     n++;
34 }
35 void insertAt(int index, int value) {
```

Task 1 - Array Operations Code Part 2

Lab 2 - Task 1

Completed C++ code and execution evidence

```
35 void insertAt(int index, int value) {
36     if (index < 0 || index > n || n == CAPACITY) // setting capacity
37     {
38         cout << "Invalid operation\n";
39         return;
40     }
41     for (int i = n; i > index; i--) {
42         arr[i] = arr[i - 1];
43     }
44     arr[index] = value;
45     n++;
46 }
47 void deleteAt(int index) // deleteeeeeeeeeee
48 {
49     if (index < 0 || index >= n) {
50         cout << "Invalid index\n";
51         return;
52     }
53     for (int i = index; i < n - 1; i++) {
54         arr[i] = arr[i + 1];
55     }
56     n--;
57 }
58 void update(int index, int newValue) // update valuye
59 {
60     if (index < 0 || index >= n) {
61         cout << "Invalid index\n";
62         return;
63     }
64     arr[index] = newValue;
65 }
66 int main() {
67     for (int val = 60; val < 89; val += 2) {
68         insertEnd(val);
69     }
```

Completed C++ code and execution evidence

```
56 int main() {  
57     for (int val = 60; val < 89; val += 2) {  
58         insertEnd(val);  
59     }  
70     cout << "n = " << n << "\n";  
71     traverse();  
72     int index = search(65);  
73     cout << "Search for 65 => " << index << "\n";  
74     insertEnd(90);  
75     cout << "After insertEnd(90): ";  
76     traverse();  
77     insertAt(2, 62);  
78     cout << "After insertAt(2, 62): ";  
79     traverse();  
80     deleteAt(3);  
81     cout << "After deleteAt(3): ";  
82     traverse();  
83     update(1, 88);  
84     cout << "After update(1, 88): ";  
85     traverse();  
86     return 0;  
87 }
```

Completed C++ code and execution evidence

```
After removing 20: 500 50 10
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\Lab 2> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\Lab 2\" ; if ($?) { g++ exercise1.cpp -o exercise1 }
f ($?) { .\exercise1 }
n = 15
Array elements: 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88
Search for 65 => -1
After insertEnd(90): Array elements: 60 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90
After insertAt(2, 62): Array elements: 60 62 62 64 66 68 70 72 74 76 78 80 82 84 86 88 90
After deleteAt(3): Array elements: 60 62 62 66 68 70 72 74 76 78 80 82 84 86 88 90
After update(1, 88): Array elements: 60 88 62 66 68 70 72 74 76 78 80 82 84 86 88 90
PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\Lab 2>
```

Completed C++ code and execution evidence

```
exercise2.cpp > main()
1  #include <iostream>
2  using namespace std;
3  const int CAPACITY = 100;
4  int arr[CAPACITY]; // put capacity
5  int n = 0;
6  void traverse() {
7      if (n == 0) {
8          cout << "Array is empty\n";
9          return;
10     }
11     for (int i = 0; i < n; i++) {
12         cout << arr[i] << " ";
13     }
14     cout << endl;
15 }
16 // Find maximum value
17 int findMax() {
18     if (n == 0) {
19         cout << "Array is empty\n";
20         return -1;
21     }
22     int maximum = arr[0];
23     for (int i = 1; i < n; i++) {
24         if (arr[i] > maximum) {
25             maximum = arr[i];
26         }
27     }
28     return maximum;
29 }
30 // Reverse the array
31 void reverseArray() {
32     int start = 0;
33     int end = n - 1;
34     while (start < end) {
35         int temp = arr[start];
```

Completed C++ code and execution evidence

```
36         arr[start] = arr[end];
37         arr[end] = temp;
38         start++;
39         end--;
40     }
41 }
42 // Remove all occurrences of a value
43 void removeAll(int value) {
44     int newSize = 0;
45     for (int i = 0; i < n; i++) {
46         if (arr[i] != value) {
47             arr[newSize] = arr[i];
48             newSize++;
49         }
50     }
51     n = newSize;
52 }
53 int main() {
54     // main
55     arr[n++] = 10;
56     arr[n++] = 20;
57     arr[n++] = 30;
58     arr[n++] = 20;
59     arr[n++] = 500;
60     arr[n++] = 20;
61     cout << "Original array: ";
62     traverse();
63     cout << "Maximum value: " << findMax() << endl;
64     reverseArray();
65     cout << "Reversed array: ";
66     traverse();
67     removeAll(20);
68     cout << "After removing 20: ";
```

Task 2 - Program Output

Lab 2 - Task 2 Home Exercise

Completed C++ code and execution evidence

```
65     cout << "Reversed array: ";
66     traverse();
67     removeAll(20);
68     cout << "After removing 20: ";
69     traverse();
70     return 0;
71 }
```

Problems Output Debug Console **Terminal** Ports

- PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\Lab 2> cd "c:\Users\MC\Desktop\3RD SEM\DSA LAB\Lab 2\" ; if (\$?) { g++ exercise2.cpp -o exercise2 } ;
f (\$?) { .\exercise2 }
Original array: 10 20 30 20 500 20
Maximum value: 500
Reversed array: 20 500 20 30 20 10
After removing 20: 500 30 10
- PS C:\Users\MC\Desktop\3RD SEM\DSA LAB\Lab 2>

Ln 60, Col 19 Spaces: 4 UTF-8 CRLF { } C++